

Lab

Java basics — methods

Lab: Introduction to methods

Objective

The objectives of this practical session are:

- To be able to write and invoke methods that have a varying number of parameters, some of which return a value.
- To accept user input in response to a prompt and process that data further, including converting it to a different type of data.
- To create and use a new class.

Part 1 — Authoring a helper method

Step-by-step

1. Back in the **labs** project, add a new class called **Program** with a main method in the package named **lab3**.
2. Add a new method in the Program class under the main method as:
`public static int getInt(String prompt)`
3. This method has a **String** parameter called **prompt**, which it displays before getting an integer value from the user. It then returns that integer.
4. Please refer to the slides to get keyboard input:

```
Scanner s = new Scanner(System.in);  
return s.nextInt();
```

The Scanner class has to be resolved. Click on the word Scanner and press **Ctrl-1** and choose **import Scanner**.

5. Create another method called `String getString(String prompt)`.
6. This method is similar to the `getInt()` method except you should change the `s.nextInt()` to `s.nextLine()`;
7. Call both methods in the `main()` method and then print the result to test your code.

Part 2 — Performing data conversions

Objective

- Use **if** statements to handle user choices.
- Collect and validate input for a **meal selection**.
- Display a **summary of the meal** at the end.

Scenario:

A student walks through the **lunch hall** serving line. The program will:

1. ask for their **main dish** (chicken, fish, or vegetarian).
2. ask how many **roast potatoes** they want.
3. ask how many **Brussel sprouts** they want.
4. display a **summary of their meal**.

Step by step

1. Create a method called **theLunchQueue** in the Program class.
2. Call the **getString()** method to display the following:
What main dish would you like(Fish, Burgers or veg) ?
And get the answer into a variable called **mainCourse**.
3. Use the **getInt()** method to display the following prompts and capture the values in suitable variable names.
How many roast potatoes would you like?
How many Brussel Sprouts would you like?
Display the description for producing a bill – something like:
Hello, your lunch is xx with yy roast potatoes and zz Brussel sprouts.
Replacing xx, yy, and zz with actual values of course!
4. Test your code by calling **theLunchQueue()** method from main().

Part 3 — Weight conversions

1. Create a method as:

```
static void convertInputToStonesPounds(int pounds).
```

This method should:

- a) Ask the user for a total weight in pounds in main() and pass the result to this method.
- b) Display the result (stones and pounds) in this method.

Note: There are 14 pounds in a stone.

Hint: Use division (/) and modulus (%)

2. Create another method as:

```
static void convertKgsToStonesPounds(int kg).
```

- a) Ask the user for a weight in kilograms.
- b) Convert the weight and display it in stones and pounds.

Hint: 1 kilo = 2.20462 pounds

Tip: Convert the Kg to pounds and then call
convertInputToStonesPounds(int kg).

3. Test your code at each stage.

Stretch material: Part 4 — move your code to a separate class

Does every method have to be in the Program class?

In this part you'll create a new class and move all the code to that class.

1. Create a new class called **Utils** without a main() method in the **lab3** package.
2. Copy the **getInt** and **getString** methods to the Utils class.
3. Remove the **static** word from every method definition.

We'll discuss static method at a later date. The only reason why every method was static was because main() is a **static** method(), but we are now free of main()!

4. Back in the **theLunchQueue** method, create an instance of **Utils** class and use it to call the **getInt()** and **getString()** methods like:

```
Utils utils = new Utils();
```

```
int potatos = utils.getInt("How many roast potatoes would you like?");
```

5. Run the application to make sure everything works.



QA.com